

Implementasi Keamanan Informasi Pada Sistem Lamaran Magang

MUHAMMAD RESQYAN NURUL AQLI (G6401231012), MUHAMMAD RIFAT ADNAN (G6401231046),
MUHAMMAD THORIQ AZIZ (G6401231107)

Abstrak

Program magang merupakan kegiatan akademik yang bertujuan memberikan pengalaman kerja nyata kepada mahasiswa serta menerapkan pengetahuan yang diperoleh selama perkuliahan. Namun, proses pengelolaan magang masih menghadapi kendala karena informasi lowongan, dokumen pendaftaran, pencatatan aktivitas, dan pelaporan kegiatan belum terintegrasi secara optimal. Selain itu, dokumen digital seperti laporan magang, transkrip nilai, dan dokumen pendukung memiliki risiko terhadap perubahan data dan akses yang tidak berwenang. Penelitian ini bertujuan untuk merancang sistem informasi manajemen magang mahasiswa berbasis web dengan menerapkan mekanisme keamanan informasi.

Metode yang digunakan adalah perancangan sistem menggunakan arsitektur client-server dengan pengembangan fitur pengelolaan lowongan, pendaftaran, dokumen, logbook, dan penilaian, serta penerapan mekanisme perlindungan data melalui enkripsi, autentikasi, pengaturan hak akses, dan pencatatan aktivitas pengguna. Hasil penelitian menunjukkan bahwa sistem yang dirancang mampu mengintegrasikan proses pengelolaan magang dan mendukung keamanan data melalui penerapan aspek kerahasiaan, integritas, ketersediaan, autentikasi, otorisasi, dan akuntabilitas.

Kata Kunci: autentikasi, integritas data, keamanan informasi, magang mahasiswa, sistem informasi, web

PENDAHULUAN

Program magang merupakan salah satu kegiatan akademik yang bertujuan untuk memberikan pengalaman kerja nyata kepada mahasiswa di dunia industri. Melalui kegiatan magang, mahasiswa dapat mengembangkan kemampuan profesional, memahami lingkungan kerja, serta menerapkan ilmu yang diperoleh selama proses perkuliahan. Namun, dalam pelaksanaannya pengelolaan kegiatan magang masih menghadapi beberapa kendala, seperti proses pencarian informasi, pengumpulan dokumen, monitoring aktivitas, dan pelaporan kegiatan yang masih dilakukan melalui beberapa media sehingga belum terintegrasi secara optimal.

Perkembangan teknologi informasi mendorong penggunaan sistem informasi berbasis web sebagai solusi untuk meningkatkan efektivitas pengelolaan data dan layanan akademik. Sistem informasi dapat membantu proses pengolahan data menjadi informasi yang lebih terstruktur sehingga mendukung aktivitas operasional suatu organisasi (Jogiyanto, 2005). Penelitian mengenai sistem informasi magang berbasis web menunjukkan bahwa penerapan sistem terintegrasi dapat membantu proses pendaftaran, pengelolaan data peserta, monitoring kegiatan, serta mengurangi permasalahan administrasi yang sebelumnya dilakukan secara manual (Ilham & Belluano, 2022).

Penelitian lain terkait sistem monitoring magang berbasis web juga menunjukkan bahwa sistem informasi dapat membantu proses pemantauan aktivitas mahasiswa, pengelolaan laporan kegiatan, serta meningkatkan transparansi antara mahasiswa dan pembimbing (Alfirmansyah, 2025). Namun, sebagian sistem informasi magang masih berfokus pada aspek pengelolaan data dan belum banyak memperhatikan aspek keamanan informasi, terutama dalam perlindungan dokumen digital seperti laporan magang, transkrip nilai, dan dokumen pendaftaran.

Keamanan informasi menjadi aspek penting dalam pengembangan sistem berbasis digital karena data yang dikelola memiliki risiko terhadap akses tidak sah, perubahan data, maupun kebocoran informasi. Menurut (Stallings 2017), keamanan sistem informasi mencakup aspek kerahasiaan, integritas, dan ketersediaan data yang bertujuan untuk menjaga

informasi agar tetap terlindungi. Penerapan mekanisme keamanan seperti enkripsi, autentikasi pengguna, serta pengaturan hak akses dapat membantu meningkatkan perlindungan terhadap data digital.

Berdasarkan penelitian terdahulu tersebut, diperlukan pengembangan Sistem Informasi Manajemen Magang Mahasiswa berbasis web yang tidak hanya mendukung proses administrasi magang, tetapi juga menerapkan mekanisme keamanan informasi. Tujuan penelitian ini adalah merancang sistem yang mampu mengintegrasikan proses pencarian lowongan, pendaftaran, pengelolaan dokumen, monitoring aktivitas, dan penilaian magang dengan menerapkan mekanisme keamanan untuk menjaga kerahasiaan, integritas, ketersediaan, autentikasi, otorisasi, dan akuntabilitas data.

METODE

Bab ini berisi ruang lingkup penelitian (jika ada) dan diikuti dengan metode yang akan digunakan. Metode merupakan langkah-langkah yang jelas sehingga pembaca dapat berhasil mengulang kembali penelitian sesuai dengan langkah yang dikemukakan. Metode yang sudah umum digunakan (seperti metode pengembangan perangkat lunak *waterfall*, *object oriented*) tidak diuraikan dengan terperinci, cukup secara singkat. Metode yang belum umum atau baru hendaklah diuraikan dengan lengkap.

A. Alur Perancangan Sistem Keamanan

Perancangan keamanan pada Sistem Informasi Magang IPB dilakukan secara sistematis melalui pendekatan siklus hidup pengembangan sistem yang aman (*Secure Software Development Life Cycle*). Alur metodologi perancangan ini bertujuan untuk memastikan setiap celah keamanan diidentifikasi sejak tahap analisis hingga tahap evaluasi kinerja sistem.

Secara umum, tahapan perancangan keamanan sistem digambarkan melalui alur berikut:

1. Analisis Kebutuhan Keamanan

Tahap awal dilakukan dengan memetakan proses bisnis utama aplikasi—seperti registrasi mahasiswa, login, penginputan logbook harian, pengunggahan draf dokumen, serta proses persetujuan berkas oleh dosen pembimbing. Analisis ini mendefinisikan aset penting yang harus dilindungi (misalnya data kredensial pengguna dan integritas dokumen rekomendasi).

2. Pemodelan Ancaman

Setelah proses bisnis dipetakan, dilakukan identifikasi potensi ancaman siber menggunakan metode *STRIDE* (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, dan Elevation of Privilege*). Melalui analisis pemodelan ancaman ini, ditentukan titik-titik rawan pada arsitektur sistem, seperti:

- Kerawanan pencurian kredensial di database (Information Disclosure).
- Kerawanan manipulasi dokumen rekomendasi yang diunggah (Tampering).
- Kerawanan akses liar ke endpoint sensitif (Elevation of Privilege).

3. Perancangan Arsitektur Keamanan

Tahap ini merancang solusi arsitektur keamanan untuk memitigasi risiko dari hasil pemodelan ancaman. Komponen keamanan yang dirancang meliputi:

- Sistem autentikasi berbasis *stateless session* menggunakan JWT (JSON Web Token).
- Mekanisme otorisasi dinamis menggunakan konsep RBAC (Role-Based Access Control).
- Modul kriptografi simetris menggunakan algoritma Bcrypt untuk keamanan kata sandi dan HMAC-SHA256 untuk tanda tangan digital dokumen.
- Penggunaan protokol pengaman jaringan (HTTPS/TLS) untuk mengamankan data yang ditransmisikan.

4. Implementasi Pengkodean Aman

Menerjemahkan rancangan keamanan ke dalam baris kode program di sisi backend menggunakan framework FastAPI. Tahap ini fokus pada implementasi modularitas kode (pemisahan layer router, service, dan repositori) guna mengisolasi logika akses data dan meminimalisasi celah keamanan seperti SQL injection serta kegagalan otorisasi.

5. Pengujian Keamanan dan Sistem

Melakukan verifikasi fungsionalitas sistem keamanan secara otomatis menggunakan *framework pytest*. Pengujian mencakup uji coba kegagalan akses token, bypass hak akses role, manipulasi berkas tanda tangan digital, serta ketahanan server terhadap serangan *brute force login* melalui pembatasan trafik.

6. Analisis dan Evaluasi Kinerja

Tahap akhir dilakukan dengan mengukur dampak performa setelah mekanisme keamanan diterapkan pada sistem. Pengukuran ini mencakup analisis *response time*, latensi jaringan, ukuran header transaksi, serta utilisasi memori dan CPU pada server backend.

B. Metode Implementasi Pengamanan Data dan Akses

Mekanisme pertahanan keamanan sistem diimplementasikan pada tingkat aplikasi (backend FastAPI) dan database. Metode pengamanan ini mencakup lima pilar utama untuk menjaga aspek kerahasiaan (*confidentiality*), integritas (*integrity*), dan ketersediaan (*availability*).

1. Autentikasi Pengguna

Proses autentikasi didesain untuk memverifikasi identitas pengguna secara aman sebelum mereka dapat berinteraksi dengan sistem. Metode ini menggunakan dua komponen utama:

- Hashing Kata Sandi: Kata sandi pengguna tidak disimpan secara langsung dalam bentuk teks biasa di database. Sistem menggunakan algoritma Bcrypt (fungsi `'bcrypt.hashpw'` dan salt dinamis dari `'bcrypt.gensalt'`) untuk mengubah kata sandi menjadi representasi hash satu arah. Sebelum proses hashing, panjang kata sandi dipotong secara eksplisit maksimal 70 karakter guna mengantisipasi keterbatasan panjang input internal Bcrypt sebesar 72 byte yang rentan terhadap pemotongan pasif.
- Token Akses: Setelah proses pencocokan hash kata sandi pada login berhasil, server akan menghasilkan token JWT yang ditandatangani menggunakan algoritma simetris HMAC-SHA256 (HS256) dengan kunci rahasia yang kuat. Token tersebut memiliki waktu kedaluwarsa selama 1440 menit (24 jam) untuk sesi normal, dan 15 menit untuk token pemulihan kata sandi.

2. Otorisasi Berbasis Peran (Role-Based Access Control - RBAC)

Setelah pengguna terautentikasi, hak akses mereka dibatasi berdasarkan peran masing-masing pengguna (mahasiswa, dosen, dan staff). Metode ini diimplementasikan melalui:

- Kelas Pendefinisian Peran: Menggunakan class dependensi FastAPI (`'RoleChecker'`) yang secara dinamis memeriksa klaim peran yang tersimpan dalam payload JWT pengguna.
- Isolasi Endpoint: Setiap rute API dilindungi dengan dependensi peran yang sesuai. Sebagai contoh, endpoint untuk mengajukan surat rekomendasi hanya dapat diakses oleh peran mahasiswa, sedangkan penilaian laporan magang hanya dapat diakses oleh peran dosen. Jika ada upaya akses lintas peran, middleware akan langsung membatalkan request dan mengembalikan pengecualian HTTP `'403 Forbidden'`.

3. Perlindungan Data at Rest dan Data in Transit

Perlindungan data dilakukan pada dua kondisi data yang berbeda:

- Data at Rest (Data Tersimpan): Selain hashing kata sandi di tingkat aplikasi, pada server database PostgreSQL cloud (Supabase) diterapkan enkripsi tingkat disk (*Transparent Data*

Encryption) menggunakan algoritma standar AES-256 yang dikelola secara aman untuk mencegah kebocoran data jika media penyimpanan fisik diekstraksi.

2. Data in Transit (Data dalam Perjalanan): Seluruh lalu lintas data antara peramban web klien dan backend server diamankan menggunakan protokol HTTPS (TLS 1.2/1.3). Selain itu, pertukaran data surat elektronik untuk reset password dilindungi oleh enkripsi transportasi STARTTLS pada port SMTP 587.

4. Integritas Dokumen (*Digital Signature*)

Untuk menjamin keaslian dan integritas berkas surat rekomendasi yang diunggah oleh mahasiswa maupun berkas bertanda tangan yang disetujui dosen, sistem menyematkan tanda tangan digital berbasis HMAC-SHA256.

Proses penandatanganan dilakukan dengan menggabungkan nama penandatanganan, ukuran file, dan seluruh byte konten berkas fisik PDF, kemudian di-hash menggunakan kunci rahasia digital signature server ('DIGITAL_SIGNATURE_SECRET'). Nilai hash signature yang dihasilkan kemudian ditempelkan secara dinamis pada query parameter tautan dokumen publik (sebagai parameter 'sig' dan 'signer'). Saat dokumen diverifikasi, backend akan mencocokkan kembali byte file terhadap parameter signature tersebut untuk memastikan tidak ada perubahan isi dokumen.

5. Pengendalian Lalu Lintas Data (*Rate Limiting*)

Sistem mengimplementasikan middleware pembatasan trafik untuk melindungi endpoint sensitif seperti login dan pendaftaran dari serangan brute-force serta penimbunan request.

Metode ini bekerja dengan mencatat alamat IP klien dan waktu akses dalam *sliding window* di memori server. Jika request dari IP tertentu melebihi batas limit per menit yang ditentukan, middleware akan memotong request secara otomatis dan mengembalikan respon HTTP '429 Too Many Requests'.

C. Metode Pengujian Sistem dan Keamanan

Metode pengujian dalam proyek ini dilakukan untuk memverifikasi kebenaran logika fungsionalitas bisnis sekaligus menguji ketahanan mekanisme keamanan yang telah dibangun. Pengujian dilakukan secara terprogram dan otomatis menggunakan kerangka pengujian (*testing framework*) pytest di lingkungan lokal sebelum sistem dideploy ke server produksi. Metodologi pengujian dirancang dengan mengklasifikasikan pengujian ke dalam tiga skenario utama:

1. Pengujian Fungsional (*Functional Testing*):

Skenario ini bertujuan memastikan seluruh alur bisnis berjalan sesuai skema. Kasus uji fungsionalitas meliputi pengujian proses registrasi akun baru, validasi kredensial login untuk menghasilkan token akses, pembuatan entri logbook baru oleh mahasiswa, pengajuan draf surat rekomendasi, serta pengiriman notifikasi antar user. Pengujian ini mensimulasikan input data positif (benar) dan negatif (salah) untuk melihat respon valid dari backend.

2. Pengujian Keamanan (*Security Testing*):

Skenario pengujian keamanan dirancang secara khusus untuk memverifikasi keandalan pertahanan aplikasi terhadap serangan siber umum:

- Pengujian Autentikasi: Menguji apakah endpoint terproteksi menolak request yang dikirimkan tanpa menyertakan token JWT, menggunakan token kedaluwarsa, atau menggunakan token dengan format manipulasi, yang diharapkan mengembalikan respon HTTP 401 Unauthorized.
- Pengujian Otorisasi (RBAC & *Isolation*): Menguji apakah pengguna dengan peran tertentu diblokir saat mencoba mengakses rute API peran lain (misalnya akun staff mencoba mengajukan surat rekomendasi mahasiswa), yang diharapkan mengembalikan respon HTTP 403 Forbidden. Pengujian juga memverifikasi bahwa

data antar dosen terisolasi dengan aman (Dosen A tidak boleh memproses surat yang ditujukan untuk Dosen B).

- **Pengujian Pembatasan Trafik (*Rate Limiting*):** Mensimulasikan serangan brute-force login secara terprogram dengan mengirimkan request berturut-turut dalam durasi singkat yang melewati ambang batas maksimal request per menit. Kasus uji ini memverifikasi bahwa server merespon dengan status HTTP 429 Too Many Requests untuk menghentikan serangan.

3. Pengujian Integrasi Dokumen (*Integration Testing*):

Skenario pengujian integrasi memverifikasi keterhubungan komponen backend dengan Supabase Cloud Storage dan modul tanda tangan digital. Pengujian memastikan berkas yang dikirimkan klien divalidasi format ekstensinya (wajib PDF dengan ukuran maksimal 5MB). Pengujian ini juga memverifikasi keabsahan nilai signature HMAC-SHA256 yang ditempelkan pada URL publik dokumen ketika diunggah kembali ke sistem.

D. Metode Analisis Kinerja Sistem

Metode analisis kinerja dilakukan untuk mengukur secara kuantitatif pengaruh atau trade-off dari penerapan fitur keamanan terhadap efisiensi dan performa operasional sistem informasi. Pengujian ini penting untuk memverifikasi bahwa penguatan keamanan tidak mengorbankan kenyamanan interaksi pengguna secara berlebihan.

Analisis dilakukan dengan membandingkan parameter kinerja sistem pada dua kondisi, yaitu kondisi sebelum fitur keamanan diaktifkan dan kondisi setelah seluruh fitur keamanan (otentikasi, otorisasi, enkripsi, digital signature, dan logging) diterapkan.

Parameter-parameter kinerja yang diukur meliputi:

1. Waktu Tanggap (*Response Time*):

Mengukur waktu total yang dibutuhkan oleh server backend dari menerima request hingga selesai mengirimkan respon kembali ke klien. Pengukuran ini difokuskan pada endpoint login (yang menjalankan komputasi Bcrypt) dan endpoint umum lainnya (yang menjalankan validasi token JWT) untuk mengetahui besarnya delay komputasi yang ditimbulkan oleh algoritma keamanan.

2. Latensi Jaringan (*Network Latency*):

Mengukur waktu tunda transmisi data pada jaringan. Analisis ini membandingkan latensi jabat tangan (*handshake latency*) koneksi HTTP biasa dengan koneksi HTTPS yang menggunakan protokol TLS 1.2/1.3 untuk mengetahui overhead enkripsi saat transit.

3. Ukuran Data Transaksi (*Data Size*):

Mengambil sampel dan membandingkan ukuran payload serta header HTTP request/response. Analisis difokuskan pada penambahan beban ukuran data (dalam satuan byte) akibat penyematannya Bearer Token JWT pada header setiap kali request API terproteksi dilakukan oleh klien.

4. Penggunaan Sumber Daya Server (*Resource Utilization*):

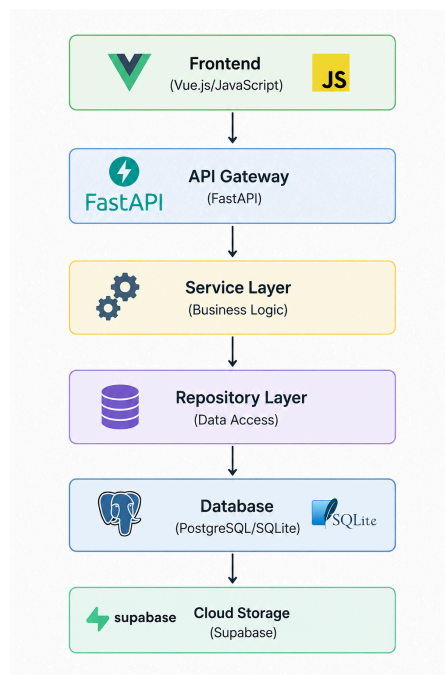
Memantau persentase utilisasi *Central Processing Unit* (CPU) dan penggunaan *Random Access Memory* (RAM) pada server backend FastAPI. Pemantauan ini dilakukan secara real-time saat server menerima beban request login serentak untuk mengevaluasi dampak pengolahan algoritma Bcrypt (*work factor*) yang memakan daya komputasi CPU yang cukup tinggi.

Alat dan Lingkungan Pengujian:

Pengukuran parameter kinerja dilakukan menggunakan logger bawaan server (Uvicorn stdout logs) yang mencatat waktu respon setiap transaksi, serta pemantauan konsol resource menggunakan tools pemantau sistem (system resource monitor) pada lingkungan server lokal dan lingkungan cloud deployment (dashboard Railway).

HASIL DAN PEMBAHASAN

Arsitektur Sistem



Gambar 1: Arsitektur Sistem

Teknologi yang Digunakan:

- Backend: FastAPI 0.110.0, Python dengan uvicorn
- Database: PostgreSQL/SQLite dengan SQLAlchemy ORM
- Authentication: JWT (PyJWT), OAuth2
- Cloud Storage: Supabase
- Frontend: Vue.js/JavaScript dengan Vite
- Security: bcrypt, HMAC-SHA256 untuk digital signature

Implementasi Pengelolaan Data Magang

Model Data (Entity-Relationship), sistem mengimplementasikan 6 entitas utama dengan relasi yang terintegrasi:

1. User (dengan Inheritance Pattern)

- Superclass: User (base untuk semua pengguna)
- Subclass:
 - Mahasiswa: NIM, Fakultas, Prodi, Dosen Pembimbing
 - Dosen: NIP, relasi dengan laporan dan logbook
 - Staff: NIP, pengelola proses seleksi
- Implementasi menggunakan Joined Table Inheritance di SQLAlchemy untuk memisahkan data user generik dengan atribut spesifik role.

2. Lowongan Magang

- Atribut: lowongan_id, perusahaan, judul_posisi, deskripsi, kualifikasi, deadline, is_active
- Mendukung filtering lowongan aktif berdasarkan deadline

3. Pendaftaran

- Menghubungkan mahasiswa dengan lowongan
- Status: PENDING, ACCEPTED, REJECTED
- Menyimpan dokumen CV dan surat rekomendasi

4. Laporan Magang

- Status: ONGOING, PENDING (menunggu penilaian), APPROVED, REVISION_NEEDED
- Menyimpan nilai, catatan dosen, dan dokumen laporan akhir
- Terintegrasi dengan logbook untuk tracking aktivitas

5. Logbook Magang

- Pencatatan aktivitas harian dengan durasi otomatis
- Atribut: tanggal_log, waktu_mulai, waktu_selesai, keterangan, dokumentasi
- Relasi dengan laporan untuk agregasi kegiatan

6. Surat Rekomendasi

- Workflow permohonan tanda tangan dosen
- Status: DRAFT, SUBMITTED, SIGNED
- Dokumen tersimpan di cloud storage

Implementasi Mekanisme Keamanan Informasi

A. Kerahasiaan (Confidentiality)

1. Password Hashing dengan Bcrypt

```
def get_password_hash(password: str) -> str:
    teks_password = str(password)[:70].encode("utf-8")
    hashed = bcrypt.hashpw(teks_password, bcrypt.gensalt())
    return hashed.decode("utf-8")
```

Gambar 2: Implementasi Kode Hash

- Password tidak pernah disimpan dalam plain text
- Menggunakan bcrypt dengan salt generation otomatis
- Validasi panjang password maksimal 70 karakter untuk keamanan

2. Autentikasi JWT (JSON Web Token)

```
def buat_access_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_SECONDS)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, get_jwt_secret_key(), algorithm=settings.ALGORITHM)
    return encoded_jwt
```

Gambar 3: Implementasi Kode Auth Token

- Token berisi: username, role, user_id, dan timestamp expiry
- Implementasi token expiry untuk membatasi window keamanan
- Algoritma HS256 atau RS256 sesuai konfigurasi

3. Reset Password dengan Token Terbatas Waktu

```
async def lupa_password(self, email: str):
    reset_token_expires = timedelta(minutes=15)
    reset_token = buat_access_token(
        data={"sub": user.username, "type": "reset"},
        expires_delta=reset_token_expires
    )
    await kirim_email_reset_password(email, reset_token)

def buat_access_token(data: dict,
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
```

Gambar 4: Implementasi Kode Reset Password

- Token reset password berlaku hanya 15 menit
- Mencegah penggunaan link lama jika terjadi kebocoran

B. Autentikasi & Otorisasi (Authentication & Authorization)

1. Role-Based Access Control (RBAC)

```
class RoleChecker:
    """Class untuk membatasi akses endpoint berdasarkan role."""
    def __init__(self, allowed_roles: list):
        self.allowed_roles = [role.lower() for role in allowed_roles]

    def __call__(self, current_user: dict = Depends(get_current_user)):
        user_role = str(current_user["role"]).lower()
        if user_role not in self.allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Akses ditolak. Endpoint ini hanya diperuntukkan bagi role: {'', '
            )
        return current_user
```

Gambar 5: Implementasi Kode RBAC

- Setiap endpoint dapat membatasi akses berdasarkan role pengguna
- Role yang didukung: MAHASISWA, DOSEN, STAFF, ADMIN
- Validasi dilakukan di tingkat endpoint dependency

2. OAuth2 Bearer Token

- Menggunakan FastAPI OAuth2PasswordBearer
- Token dikirim via header Authorization: Bearer <token>
- Validasi dilakukan pada setiap request

3. Kontrol Akses Tingkat Data

```
def ubah_laporan(self, laporan_id: int, data: LaporanUpdate, user_id: int):
    laporan = self.repo.get_by_id(laporan_id)
    if laporan.mahasiswa_id != user_id:
        raise HTTPException(status_code=403, detail="Akses ditolak.")
```

Gambar 6: Implementasi Kode Kontrol Akses Data

- Mahasiswa hanya dapat mengakses data milik mereka sendiri
- Dosen hanya dapat menilai laporan yang ditugaskan
- Staff dapat melihat semua data untuk proses seleksi

C. Integritas Data (Data Integrity)

1. Digital Signature dengan HMAC-SHA256

```
def generate_signature(file_bytes: bytes, signer_name: str) -> str:
    """Buat tanda tangan HMAC-SHA256 sederhana untuk dokumen."""
    payload = f"{signer_name}:{len(file_bytes)}:".encode("utf-8") + file_bytes
    return hmac.new(DEFAULT_SIGNATURE_SECRET.encode("utf-8"), payload, hashlib.sha256).hexdigest()

def verify_signature(file_bytes: bytes, signer_name: str, signature: str) -> bool:
    """Verifikasi tanda tangan yang dihasilkan dari isi file dan nama penandatangan."""
    return hmac.compare_digest(generate_signature(file_bytes, signer_name), signature)
```

Gambar 7: Implementasi Kode DigiSing

- Setiap dokumen yang diupload mendapat signature unik
 - Signature melekat pada URL dokumen di Supabase
 - Deteksi modifikasi dokumen melalui verifikasi ulang signature
- ### 2. Cloud Storage Encryption (Supabase)

- Semua file tersimpan terenkripsi di Supabase Storage
 - Folder terorganisir: laporan/, dokumentasi/, pendaftaran/cv/, surat_rekomendasi/
 - Public URL dengan signature parameter untuk verifikasi integritas
3. ORM Query Protection
- Menggunakan SQLAlchemy ORM untuk mencegah SQL Injection
 - Parameter binding otomatis untuk semua query database
 - Tidak ada raw SQL strings dalam aplikasi

D. Ketersediaan (Availability)

1. Rate Limiting

```
RATE_LIMIT_REQUESTS_PER_MINUTE = 60
RATE_LIMIT_WINDOW_SECONDS = 60
RATE_LIMIT_EXEMPT_PATHS = ("/", "/docs", "/openapi.json", "/redoc")

class RateLimitMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        # Prune stale timestamps dan cek limit
        if len(timestamps) >= RATE_LIMIT_REQUESTS_PER_MINUTE:
            return JSONResponse(
                status_code=429,
                content={"detail": "Too many requests. Please try again later."},
            )
```

Gambar 8: Implementasi Kode Rate Limiting

- Membatasi 60 request per menit per client IP
 - Middleware mencegah DDoS dan abuse
 - Per-path rate limiting untuk granularity lebih baik
2. Database Connection Pooling
- SQLAlchemy connection pool untuk reuse koneksi
 - Mengurangi overhead koneksi database
3. Async Processing untuk Background Tasks
- Email notifikasi diproses di background untuk responsivitas API

E. Akuntabilitas (Accountability)

1. Sistem Notifikasi Terintegrasi

```
self.notif_repo.create({
    "user_id": mahasiswa.user_id,
    "isi_notifikasi": f"Laporan magang Anda telah dinilai oleh {dosen.nama}...",
    "target_url": f"/mahasiswa/laporan/{laporan_id}"
})
```

Gambar 9: Implementasi Kode Notifikasi

- Setiap action penting (submit, approve, reject) membuat notifikasi
 - User dapat melacak history aktivitas mereka
 - Timestamp otomatis pada setiap notifikasi
2. Email Audit Trail
- Semua email notifikasi terkirim tercatat di database
 - Template email mengandung detail lengkap action
 - Dapat digunakan untuk audit trail perubahan data
3. Metadata Tracking
- Setiap record menyimpan timestamp created_at dan updated_at
 - Dosen_id dan user_id tercatat untuk setiap action perubahan
 - Status history dapat direkonstruksi

SIMPULAN

Berdasarkan hasil perancangan sistem, dapat disimpulkan bahwa Sistem Informasi Manajemen Magang Mahasiswa berbasis web dapat menjadi solusi untuk mengintegrasikan proses pengelolaan kegiatan magang, mulai dari pencarian lowongan, pendaftaran, pengelolaan dokumen, pencatatan aktivitas melalui logbook, hingga proses evaluasi dan penilaian. Sistem ini mampu membantu mahasiswa, dosen, dan pihak akademik dalam melakukan pengelolaan serta monitoring kegiatan magang secara lebih efektif.

Penerapan aspek keamanan informasi pada sistem dapat meningkatkan perlindungan terhadap data dan dokumen digital yang dikelola. Mekanisme keamanan yang diterapkan mencakup pengelolaan hak akses pengguna, autentikasi, pencatatan aktivitas, serta perlindungan terhadap integritas dan kerahasiaan data. Dengan adanya sistem ini, proses pengelolaan magang diharapkan menjadi lebih terstruktur, aman, dan mudah dipantau.

Pengembangan selanjutnya dapat dilakukan dengan meningkatkan fitur keamanan, seperti penerapan mekanisme verifikasi dokumen yang lebih kompleks, penguatan sistem audit, serta pengujian keamanan untuk mengetahui tingkat ketahanan sistem terhadap berbagai ancaman.

DAFTAR PUSTAKA

Ilham, M., & Belluano, P. L. L. (2022). Sistem Informasi Magang Bersertifikat Berbasis Web. *Indonesian Journal of Data and Science*, 3(2), 69–76.

Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson Education

Kurniawan, A., & Putra, R. (2021). Pengembangan Sistem Informasi Manajemen Magang Berbasis Web untuk Mendukung Proses Administrasi Mahasiswa. *Jurnal Teknologi Informasi dan Komputer*